

PROGRAM 20:

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~
cd OSSP
vedsk0831@DESKTOP-83FU7HB:~/OSSP$ pwd
/home/vedsk0831/OSSP
vedsk0831@DESKTOP-83FU7HB:~/OSSP$ mkdir -p Program20
cd Program20
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program20$ pwd
/home/vedsk0831/OSSP/Program20
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program20$ nano program20.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program20$ gcc -Wall -Wextra program20.c -o program20
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program20$ ls -l
total 28
-rwxr-xr-x 1 vedsk0831 vedsk0831 17072 Oct  6 13:43 program20
-rw-r--r-- 1 vedsk0831 vedsk0831  5301 Oct  6 13:43 program20.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program20$ ./program20
=====
  OSSP PROGRAM 20
  Process Groups and Terminal Control
=====
Shell PID   : 1870
Shell PGID  : 1870

Commands:
  Any Linux command
  exit

shell> echo hello
Child PID   : 1871
Child PGID  : 1871
hello
Child exited normally.
shell> sleep 10
Child PID   : 1872
```

```
Commands:
  Any Linux command
  exit

shell> echo hello
Child PID   : 1871
Child PGID  : 1871
hello
Child exited normally.
shell> sleep 10
Child PID   : 1872
Child PGID  : 1872
Child exited normally.
shell> sleep 30
Child PID   : 1875
Child PGID  : 1875
^CChild terminated by signal 2
shell> echo Terminal_Control_Restored
Child PID   : 1876
Child PGID  : 1876
Terminal_Control_Restored
Child exited normally.
shell> |
```

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <signal.h>

#include <sys/wait.h>

#include <termios.h>


#define MAX_ARGS 64


pid_t shell_pgid;

pid_t foreground_pgid = -1;
```

```

/* Ctrl+C handler */
void handle_sigint(int sig)
{
    (void)sig;

    if (foreground_pgid > 0) {
        printf("\nShell: forwarding SIGINT\n");
        kill(-foreground_pgid, SIGINT);
    }
}

/* Ctrl+Z handler */
void handle_sigtstp(int sig)
{
    (void)sig;

    if (foreground_pgid > 0) {
        printf("\nShell: forwarding SIGTSTP\n");
        kill(-foreground_pgid, SIGTSTP);
    }
}

int main()
{
    char command[256];

    /* Get shell PID */

```

```

shell_pgid = getpid();

/*
 * Ignore terminal control signals.
 * This prevents the shell from being stopped.
 */
signal(SIGTTOU, SIG_IGN);
signal(SIGTTIN, SIG_IGN);
signal(SIGTSTP, SIG_IGN);

/* Create shell process group */
setpgid(shell_pgid, shell_pgid);

/* Give terminal to shell */
tcsetpgrp(STDIN_FILENO, shell_pgid);

/* Register signal handlers */
signal(SIGINT, handle_sigint);
signal(SIGTSTP, handle_sigtstp);

printf("=====\n");
printf(" OSSP PROGRAM 20\n");
printf(" Process Groups and Terminal Control\n");
printf("=====\n");

printf("Shell PID : %d\n", shell_pgid);
printf("Shell PGID : %d\n", getpgrp());

```

```

printf("\nCommands:\n");

printf(" Any Linux command\n");

printf(" exit\n\n");

while (1) {

    printf("shell> ");

    fflush(stdout);

    if (fgets(command,
                sizeof(command),
                stdin) == NULL) {
        break;
    }

    /* Remove newline */
    command[strcspn(command, "\n")] = '\0';

    /* Ignore empty command */
    if (strlen(command) == 0)
        continue;

    /* Exit */
    if (strcmp(command, "exit") == 0)
        break;

    /*
     * Copy command before strtok()

```

```
*/  
  
char command_copy[256];  
strcpy(command_copy, command);  
  
/* Create argument array */  
char *args[MAX_ARGS];  
  
int argc = 0;  
  
/* Split command */  
char *token = strtok(command_copy, " ");  
  
while (token != NULL &&  
       argc < MAX_ARGS - 1) {  
  
    args[argc++] = token;  
  
    token = strtok(NULL, " ");  
}  
  
args[argc] = NULL;  
  
if (argc == 0)  
    continue;  
  
/* Create child */  
pid_t pid = fork();
```

```
if (pid < 0) {  
    perror("fork");  
    continue;  
}
```

```
/* CHILD */
```

```
if (pid == 0) {
```

```
    /*  
     * Create a new process group.  
     */  
    setpgid(0, 0);
```

```
    /*  
     * Restore default signal behavior.  
     */  
    signal(SIGINT, SIG_DFL);  
    signal(SIGTSTP, SIG_DFL);  
    signal(SIGTTIN, SIG_DFL);  
    signal(SIGTTOU, SIG_DFL);
```

```
    /*  
     * Execute command.  
     */  
    execvp(args[0], args);
```

```
    perror("execvp");  
    exit(1);
```

```
}
```

```
/* PARENT */
```

```
else {
```

```
/*
```

```
 * Put child in its own process group.
```

```
*/
```

```
setpgid(pid, pid);
```

```
foreground_pgid = pid;
```

```
printf("Child PID : %d\n", pid);
```

```
printf("Child PGID : %d\n", getpgid(pid));
```

```
/*
```

```
 * Transfer terminal control
```

```
 * to child process group.
```

```
*/
```

```
tcsetpgrp(STDIN_FILENO,
```

```
    foreground_pgid);
```

```
int status;
```

```
/*
```

```
 * Wait for child.
```

```
*/
```

```
waitpid(pid,
```

```

        &status,
        WUNTRACED);

/*
 * Give terminal back to shell.
 */
tcsetpgrp(STDIN_FILENO,
        shell_pgid);

foreground_pgid = -1;

if (WIFEXITED(status)) {

    printf("Child exited normally.\n");

} else if (WIFSIGNALED(status)) {

    printf("Child terminated by signal %d\n",
        WTERMSIG(status));

} else if (WIFSTOPPED(status)) {

    printf("Child stopped by signal %d\n",
        WSTOPSIG(status));

/*
 * Resume stopped process.
 */

```

```
kill(-pid, SIGCONT);
```

```
waitpid(pid, NULL, 0);
```

```
}
```

```
}
```

```
}
```

```
/* Restore terminal to shell */
```

```
tcsetpgrp(STDIN_FILENO, shell_pgid);
```

```
printf("\nProgram 20 terminated.\n");
```

```
return 0;
```

```
}
```